G^{3}VLA: Geometric inductive bias for Vision-Language-Action Models

Anonymous Author(s)
Affiliation
Address
email

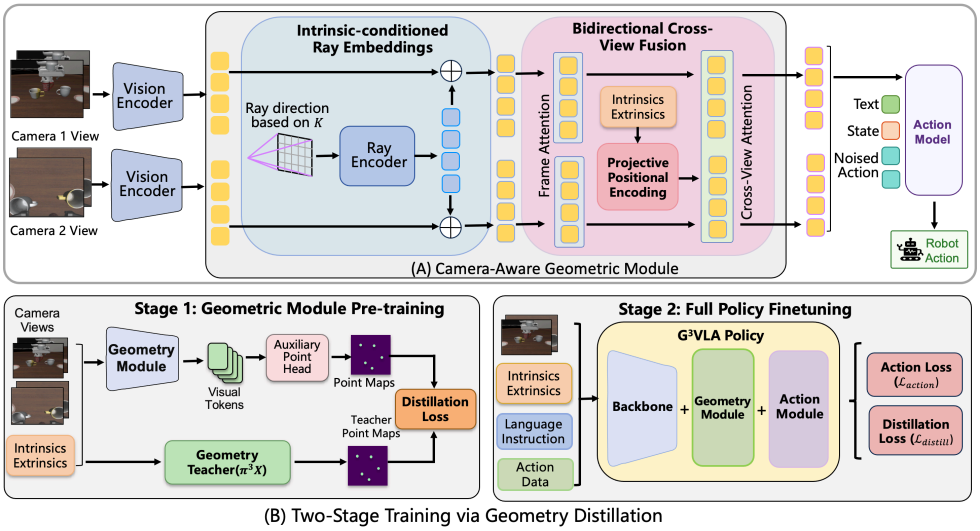


Figure 1: G^{3}VLA overview. (A) Geometric inductive bias is injected into VLA visual tokens via intrinsic-conditioned ray embeddings (K^{-1}) and bidirectional cross-view fusion with PRoPE, leaving the pretrained backbone and action objective unchanged. (B) Stage 1 distills dense point maps from π^3X to pretrain the geometry modules; Stage 2 fine-tunes the full policy under action and distillation losses jointly.

Abstract: Vision-language-action (VLA) models have made rapid progress in generalist robot manipulation by harnessing semantic knowledge from pretrained vision-language backbones, but their visual tokens remain grounded in 2D image coordinates rather than the calibrated geometry of the robot's cameras — a mismatch especially pronounced in multi-camera setups, where views are coupled by known intrinsics and extrinsics yet processed as independent images. We propose G^{3}VLA, a camera-aware geometric module that injects calibrated structure into the visual-token stream of a pretrained VLA without altering its action space or imitation objective, combining intrinsic-conditioned ray embeddings, projective positional encoding (PRoPE), and bidirectional cross-view fusion. Geometric supervision is provided either from ground-truth point maps when available, or from confidence-gated π^3X teacher predictions, requiring no depth sensors or manual annotations. Instantiated on π_0, G^{3}VLA yields consistent gains across the LIBERO suites, RoboCasa24, RoboTwin2.0, and real-robot settings, with the largest improvements on spatially and object-sensitive tasks. We further validate on $\pi_{0.5}$ and GR00T 1.5, with results suggesting that geometric transfer is most effective when geometry-aware

Submitted to the 10th Conference on Robot Learning (CoRL 2026). Do not distribute.

tokens have direct access to the action generation pathway. Our project page is at https://sites.google.com/view/g3vla.

Keywords: Vision-Language-Action, Geometric Inductive Bias, Camera Geometry

1 Introduction

Generalist robot manipulation requires two capabilities that have historically been difficult to combine: semantic understanding of instructions and objects, and spatial precision in estimating poses, distances, and cross-view geometry. Vision-Language-Action (VLA) models such as RT-2, OpenVLA, π_0, and GR00T leverage pretrained vision-language backbones to support language-conditioned control across diverse tasks [1, 2, 3, 4], but their visual interfaces remain largely 2D-image-token based, forcing calibrated camera geometry to be learned implicitly from action supervision. This limitation is especially consequential in calibrated multi-camera systems, where known intrinsics and extrinsics geometrically couple the views — structure that conventional image-token formulations leave unexploited.

Explicit geometric structure has been shown to benefit manipulation: PerAct voxelizes RGB-D observations [5], RVT aggregates multi-view renders [6], and Act3D operates in 3D feature fields [7]. These methods achieve strong spatial precision but rely on task-specific architectures that cannot leverage pretrained VLM semantics. Recent bridging efforts such as SpatialVLA [8] and 3D-VLA [9] begin to close this gap, but require explicit 3D sensor inputs, large-scale spatial pretraining, or modifications to the action representation. The key question remains: Can calibrated camera geometry serve as a lightweight visual-token pathway for pretrained VLAs, improving spatial generalization without modifying the backbone, action space, or imitation objective?

We answer this affirmatively with G^{3}VLA, a camera-aware geometric framework that injects calibrated structure directly into the visual-token stream of pretrained VLA policies without modifying the backbone or action formulation. G^{3}VLA leverages three geometric modules: intrinsic-conditioned ray embeddings [10], which tag each ViT patch token with its back-projected viewing direction from K^{-1}; projective positional encoding (PRoPE) [11], which augments rotary positional embeddings with a camera-calibrated bias encoding cross-view projective relationships; and bidirectional cross-view fusion, which exchanges geometric context across camera streams before tokens reach the action model. Geometric supervision is provided either from ground-truth point maps when depth is available, or from confidence-gated π^3X [12] distillation when only camera intrinsics and extrinsics are provided — requiring no depth sensors or manual annotations. Geometry enters the policy exclusively through the visual-token representation, leaving the pretrained backbone and action objective intact.

To evaluate whether calibrated geometry provides a transferable inductive bias beyond a single policy, we instantiate G^{3}VLA on three architecturally distinct VLA systems: π_0, $\pi_{0.5}$, and GR00T 1.5. On π_0, G^{3}VLA yields consistent improvements across LIBERO, RoboCasa24, and RoboTwin2.0, with the largest gains on spatially and object-sensitive tasks. On $\pi_{0.5}$, it further improves near-saturated LIBERO performance, confirming backbone compatibility. On GR00T 1.5, gains are mixed: its two-tower architecture — where the diffusion policy accesses visual features via cross-attention to a frozen VLM rather than consuming geometry-aware tokens directly — may attenuate the geometric signal. We treat this as suggestive evidence that the benefit of geometric injection depends on how directly geometry-aware tokens participate in action generation, though a more controlled study is needed to isolate the architectural factor.

Our contributions are as follows: (1) A geometric gap in VLAs: We identify the mismatch between 2D-grounded visual tokens in pretrained VLAs and the calibrated spatial structure required for precise manipulation. (2) G^{3}VLA: A lightweight, backbone-preserving visual-

token pathway that injects calibrated camera geometry into pretrained VLA policies via ray embeddings, PRoPE, and cross-view fusion, without modifying the action space or imitation objective. (3) Geometry distillation from π^3X: A two-stage procedure supervising geometric modules with confidence-gated dense point maps from a visual geometry teacher, requiring no manual 3D annotations. (4) Multi-architecture validation: Consistent gains on π_0 and $\pi_{0.5}$, together with an architectural analysis on GR00T 1.5 whose mixed results suggest that the benefit of geometric injection depends on how directly geometry-aware tokens reach the action pathway.

2 Related Work

Vision-language-action models. VLA models reframe language-conditioned manipulation as sequence modeling over images, text, and actions. RT-1 and RT-2 showed that transformer policies with Internet-scale vision-language pretraining transfer semantic knowledge to robot control [13, 1], and OpenVLA, π_0, and $\pi_{0.5}$ make this practical via open checkpoints, continuous-action flow matching, and co-trained open-world generalization [2, 3, 14]. These models nonetheless inherit largely 2D visual interfaces, learning camera geometry only indirectly from action supervision. Rather than scaling pretraining or altering the action decoder, we preserve the policy interface and expose calibrated camera structure to the visual tokens that drive the action model.

Structured 3D representations for manipulation. Many policies improve spatial precision through explicit 3D scene representations: PerAct uses voxelized RGB-D observations [5], RVT aggregates rendered multi-view features [6], Act3D lifts 2D features into a 3D feature field [7], PolarNet operates on point clouds [15], and 3D Diffuser Actor pairs 3D scene representations with action diffusion for stronger cross-viewpoint generalization [16]. These approaches confirm the value of geometric structure—especially for precise end-effector placement—but modify the policy interface with voxels or point clouds; we instead keep an RGB-based VLA and introduce calibration as a token-level inductive bias rather than an explicit scene representation.

Spatially aware VLAs. Other work connects VLA policies more directly to spatial representations: 3D-VLA couples action prediction with 3D world modeling [9], SpatialVLM studies metric spatial reasoning in VLMs [17], SpatialVLA introduces egocentric 3D encodings with adaptive action grids for cross-robot transfer [8], and 3DS-VLA builds a spatially aware VLA around 3D representations [18]. Unlike these, which alter the policy state or action space, we inject calibrated multi-camera geometry directly into the visual-token stream of a pretrained VLA.

Camera-aware representations and geometry distillation. Multi-camera platforms often provide calibrated intrinsics and extrinsics, yet policies typically process each stream as an ordinary image. Camera-aware positional encodings expose this structure to attention: Cameras as Relative Positional Encoding introduces PRoPE, a projective signal that captures cross-view relations through the camera model rather than learned appearance alone [11]. Complementarily, feed-forward visual-geometry models such as DUSt3R, VGGT, and π^3 predict dense point maps directly from RGB [19, 20, 12], supplying geometric targets without depth sensors. We bring both to VLA: token-level ray embeddings and attention-level projective geometry fuse views before the action module, supervised by distilled π^3X point maps tied to robot control.

3 Method

We consider language-conditioned manipulation from calibrated multi-camera observations. At each control step the policy receives a language instruction l, proprioceptive state s_t, and RGB images $\{I_t^v\}_{v=1}^V$ from V views, each with intrinsic matrix K^v and extrinsic pose T^v,

and predicts an action chunk:

$$\pi_\theta(a_{t:t+H-1} \mid l,\, s_t,\, \{I_t^v, K^v, T^v\}_{v=1}^V). \tag{1}$$

Standard VLAs discard $\{K^v, T^v\}$ and process each view independently. G^{3}VLA preserves this calibration by inserting a Camera-Aware Geometric Module into the visual-token stream before action prediction (Fig. 1), leaving the pretrained backbone, action space, and imitation objective unchanged.

3.1 Camera-Aware Geometric Module

A pretrained vision encoder [21, 22] produces P patch tokens per view, $z_p^v \in \mathbb{R}^d$, that encode 2D appearance and position but not the physical viewing direction of each patch or the geometric relationships between views. We learn a calibration-conditioned transformation that adds this structure:

$$h_{1:P}^{1:V} = F_\psi\big(z_{1:P}^{1:V},\ \{K^v, T^v\}_{v=1}^V\big)\,. \tag{2}$$

As shown in Fig. 1(A), F_ψ comprises three components. First, Intrinsic-conditioned Ray Embeddings tag each token with its back-projected viewing direction from K^v. Then, Bidirectional Cross-View Fusion exchanges geometric context across camera streams, using Projective Positional Encoding (PRoPE)—a calibration-derived attention bias—to encode cross-view projective relationships from K^v and T^v.

Intrinsic-conditioned Ray Embeddings. Under different intrinsics the same pixel (x, y) maps to a different physical viewing direction—an ambiguity that 2D positional embeddings cannot resolve. For a homogeneous pixel coordinate $u = (x, y, 1)^\top$, the normalized projection ray is $\tilde{r}^{\,v}(u) = (K^v)^{-1}u$. Since the pinhole model fixes the third coordinate, the first two components define an image-plane ray coordinate without assuming metric depth:

$$R^v(x, y) = \big[\tilde{r}^{\,v}(x, y, 1)\big]_{1:2} \in \mathbb{R}^2. \tag{3}$$

A learnable embedding G_ϕ projects this ray map to the patch grid and adds it to the encoder output before cross-view fusion, ensuring all downstream attention operates on intrinsic-aware tokens:

$$z_{0,p}^v = z_p^v + G_\phi(R^v)_p. \tag{4}$$

Projective Positional Encoding (PRoPE). Ray embeddings encode the viewing direction local to each camera but do not capture how tokens from different views relate geometrically. PRoPE [11] fills this gap by deriving fixed projective transforms for the query, key, and value representations from per-view intrinsics K^v, camera-to-world matrices from T^v, and patch locations—giving cross-view attention access to camera-model-based projective relations rather than relying on appearance similarity alone.

Bidirectional Cross-View Fusion. The fusion module proceeds in two steps. Frame Attention processes tokens within each camera stream independently, preserving view-local structure. Cross-View Attention then flattens view and patch dimensions so that all valid tokens attend bidirectionally across views, with PRoPE as the positional signal:

$$H = \mathrm{Fusion}_\psi\big(Z;\ \{K^v, T^v\}_{v=1}^V\big)\,, \tag{5}$$

where Z collects the ray-augmented per-view tokens and H is the fused sequence passed to the action model in the same token interface expected by the pretrained VLA.

3.2 Geometry Distillation and Two-Stage Training

The geometric module is initialized from scratch and receives only a sparse, task-level gradient from the action loss. We therefore introduce a dense auxiliary geometry distillation objective and a two-stage training curriculum to bootstrap the geometric pathway before full policy finetuning.

Geometry Distillation Objective. An auxiliary point head, attached to the fused visual tokens before VLA projection, provides dense geometric supervision. Patch tokens are reshaped to the $H_p \times W_p$ grid and decoded by a lightweight transformer followed by a convolutional upsampler, yielding per-pixel predictions $\hat{q}_u^v \in \mathbb{R}^2$ (ray coordinate) and $\hat{d}_u^v \in \mathbb{R}$ (log-z depth). The head is discarded at inference. Supervision targets q_u^v and d_u^v come from one of two sources. When ground-truth depth is available—e.g. in simulation—we set the validity mask $m_u^v = 1$ everywhere. Otherwise, we use π^3X [12] as an offline geometry teacher, obtaining per-pixel targets and confidence logits c_u^v converted to a hard gate:

$$m_u^v = \mathbf{1}[\sigma(c_u^v) > \tau], \tag{6}$$

with $\tau = 0.1$. The distillation loss is unified across both modes:

$$\mathcal{L}_{\text{distill}} = \frac{\sum_{v,u} m_u^v \left(\frac{1}{2} \|\hat{q}_u^v - q_u^v\|_2^2 + (\hat{d}_u^v - d_u^v)^2 \right)}{\sum_{v,u} m_u^v + \epsilon}. \tag{7}$$

Two-Stage Training Curriculum. We combine the action and distillation losses:

$$\mathcal{L} = \lambda_{\text{act}} \mathcal{L}_{\text{act}} + \lambda_{\text{distill}} \mathcal{L}_{\text{distill}}, \tag{8}$$

where $\mathcal{L}_{\text{act}}$ is the base VLA's action objective, left unchanged—in our π_0 instantiation, the original flow-matching loss. As illustrated in Fig. 1(B), we optimize this in two stages (hyperparameters in Appendix D). Stage 1: Geometric Module Pre-training. We update only the ray embeddings, cross-view fusion layers, and auxiliary point head, keeping the pretrained backbone frozen and the distillation loss dominant. This aligns the geometry modules with the dense teacher signal before the action objective takes over. Stage 2: Full Policy Fine-tuning. Starting from the Stage 1 checkpoint, we unfreeze all parameters and let the action loss dominate, retaining distillation as a lightweight geometric regularizer. At inference, the policy requires only RGB images, proprioceptive state, a language instruction, and camera calibration—neither π^3X nor the auxiliary head is queried.

4 Experiments

We evaluate whether calibrated camera geometry provides a useful inductive bias for pre-trained vision-language-action policies, addressing three questions: (1) Does the camera-aware geometric module improve manipulation on standard simulated VLA benchmarks? (2) Do the gains generalize beyond LIBERO-style tabletop scenes to more diverse household environments? (3) Which geometric components and supervision sources matter most? We report main simulation results on LIBERO, RoboCasa24, and RoboTwin2.0, then LIBERO ablations, and finally real-robot results.

4.1 Experimental Setup

Across backbones, geometry-module implementation, camera-geometry preprocessing, and teacher-target generation are shared (Appendices A–C); only the training hyperparameters are backbone-specific (Appendix D). The complete evaluation protocols are given in the Appendix F.

Simulation benchmarks. We evaluate G^{3}VLA in three simulation settings. LIBERO [23] is used as the primary benchmark because its four suites (LIBERO-Goal, LIBERO-Spatial, LIBERO-Object, and LIBERO-10) test complementary forms of generalization: goal variation, spatial relations, object variation, and long-horizon task composition. We further include RoboCasa24 [24], a broader household manipulation benchmark with more diverse kitchen layouts, object configurations, and task families, and the RoboTwin2.0 [25]

`handover_block` task, a targeted diagnostic for bimanual manipulation with more than two camera views. We report task success rate as the primary metric.

Base policies and variants. We evaluate G^{3}VLA across three backbones: π_0, $\pi_{0.5}$, and GR00T 1.5. π_0 is the main backbone and is evaluated on LIBERO, RoboCasa24, and RoboTwin2.0 with the original action space, policy interface, and flow-matching objective unchanged. $\pi_{0.5}$ is evaluated on LIBERO as a stronger near-saturation baseline. GR00T 1.5 is evaluated on LIBERO to assess how architectural differences affect geometric inductive bias. For supervision, we consider two supervision regimes: G^{3}VLA (GT), which uses ground-truth point-map supervision in simulation, and G^{3}VLA (π^3X), which uses confidence-gated teacher predictions when only camera intrinsics and extrinsics are available. We further report LIBERO ablations on π_0: w/o Ray (removes intrinsic-conditioned ray embeddings), w/o PRoPE (removes projective positional encoding), and 1-Stage (removes the staged training curriculum).

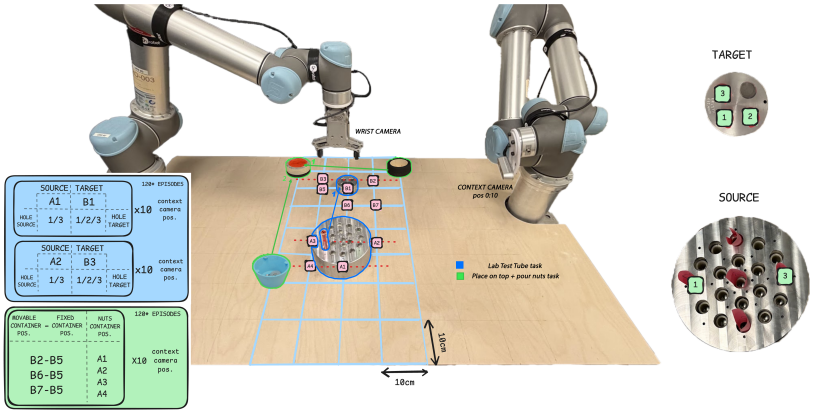


Figure 2: Real-World experimental setup on bimanual UR5 robotic arm bench. Two tasks are used for evaluation: Pick and Place Test Tube (in blue), and Pouring Nut (in green). Dataset distribution is presented in the table defined by the relevant object positions for a given task. Each combination of positions is repeated 10 times with a different context camera position.

Real-World Benchmarks. We evaluate on two tasks using a bimanual Universal Robots UR5 workbench, where one arm performs manipulation and the other holds a context camera that varies across 10 recorded positions. The two tasks are: Pick-and-Place Test Tube requires transporting a lab test tube between holders across four source-target configurations; and Pouring Nut, a long-horizon two-stage task requiring stacking a container then pouring contents into it. See Appendix F.2 for detailed description of the task and the evaluation protocol. Each dataset contains 120 episodes collected on a 10×10 cm workspace grid, which allows accurate replication of in-distribution object positions. This allows us to test the only variable of interest: camera position shifts. Camera intrinsics are captured once at initialization; extrinsics are computed per frame from forward kinematics. Training uses 10 camera viewpoints (1–10). Evaluation reports in-distribution results on views 1 and 3, and out-of-distribution generalization on unseen views 11, 12, and 13, which are excluded from training.

4.2 Main Simulation Results

LIBERO. Table 1 reports the main LIBERO results on π_0. G^{3}VLA improves over the baseline under both supervision settings. With ground-truth geometric supervision, G^{3}VLA (GT) increases the macro-average success rate from 84.6% to 88.1%, corresponding to a +3.5 point absolute gain. The largest gains occur on LIBERO-Object and LIBERO-Spatial, where success improves by +5.0 and +4.0 points respectively — consistent with

Suite	π_0 Baseline	G^{3}VLA (π^3X)	G^{3}VLA (GT)	Gain
Goal	87.4	88.4	88.4	+1.0
Spatial	85.2	88.6	89.2	+4.0
Object	89.4	93.4	94.4	+5.0
L-10	76.5	77.6	80.4	+3.9
Average	84.6	87.0	88.1	+3.5

Table 1: LIBERO results on π_0 (%). Gain denotes absolute improvement of G^{3}VLA (GT) over baseline.

Benchmark	Method	Success
RoboCasa24	π_0 Baseline	34.2
	G^{3}VLA (π^3X)	36.5
	G^{3}VLA (GT)	37.1
RoboTwin2.0	π_0 Baseline	44.0
	G^{3}VLA (π^3X)	41.0
	G^{3}VLA (GT)	49.0

Table 2: Broader results on π_0 (%). RoboCasa24: total avg.; RoboTwin2.0: handover_block.

Suite	$\pi_{0.5}$ Official	$\pi_{0.5}$ Reprod.	G^{3}VLA (π^3X)
Spatial	98.0	98.8	98.2
Object	99.0	98.4	99.0
Goal	98.2	94.4	98.0
L-10	91.4	91.8	92.8
Avg.	96.7	95.9	97.0

Table 3: LIBERO validation on $\pi_{0.5}$ (%). Official results are shown as external reference only; comparisons are made against reproduced $\pi_{0.5}$ baseline.

Suite	GR00T 1.5	G^{3}VLA (GT)	G^{3}VLA (π^3X)
Spatial	96.6	94.2	96.6
Object	97.0	97.0	99.0
Goal	95.4	94.2	95.8
L-10	90.6	92.6	89.6
Avg.	94.90	94.50	95.25

Table 4: LIBERO validation on GR00T 1.5 (%). G^{3}VLA (GT) does not improve on average, while G^{3}VLA (π^3X) gives a small gain.

the motivation of the method: tasks that require object localization and spatial relation reasoning benefit most from calibrated camera structure. G^{3}VLA (π^3X) also improves the average success rate to 87.0%, showing that teacher-predicted geometry remains useful when ground-truth supervision is unavailable.

RoboCasa24 and RoboTwin2.0. Table 2 reports results on the two broader benchmarks. On RoboCasa24, G^{3}VLA (GT) improves the total average from 34.2% to 37.1%, while G^{3}VLA (π^3X) reaches 36.5%, with gains uneven across task families (see Table 7). RoboTwin2.0 exposes a teacher-supervision failure case: G^{3}VLA (GT) improves the handover-block task from 44.0% to 49.0%, whereas G^{3}VLA (π^3X) drops to 41.0%. We attribute this gap to unreliable offline π^3X point-map targets in visually clean synthetic scenes; simulator ground-truth point maps remove this bottleneck (detailed in Appendix F.4). Thus, calibrated geometry remains beneficial when the supervision signal is reliable, while teacher-distilled geometry is sensitive to domain mismatch.

4.3 Generalization Across Backbones

Validation on a stronger backbone ($\pi_{0.5}$). Table 3 evaluates G^{3}VLA on the stronger $\pi_{0.5}$ backbone. Since the official and reproduced baselines differ slightly, we compare primarily against our reproduced $\pi_{0.5}$ baseline. G^{3}VLA-$\pi_{0.5}$ improves the macro average from 95.85% to 97.0%. The gain is smaller than on π_0, which is expected because $\pi_{0.5}$ is already near saturation on LIBERO. We view this as confirmation that the camera-aware interface remains compatible with a stronger base policy and can still provide a small additional gain.

Architectural generalization (GR00T 1.5). On GR00T 1.5's two-tower architecture, where the diffusion policy reaches visual features via cross-attention to a frozen VLM rather than consuming tokens directly, the effect is asymmetric (Table 4): G^{3}VLA (π^3X) improves the macro average from 94.90% to 95.25%, while G^{3}VLA (GT) does not (94.50%). We attribute this to the indirect pathway—geometric tokens must cross an extra attention bottleneck before

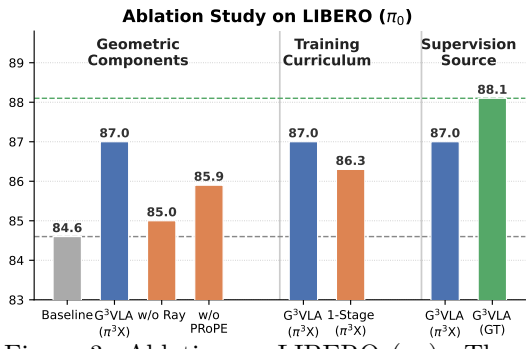


Figure 3: Ablation on LIBERO (π_0). Three axes: geometric components (ray, PRoPE), training curriculum (two-stage vs. one-stage), and supervision source (GT vs. π^3X). Dashed lines mark baseline (84.6%) and G^{3}VLA (GT) (88.1%).

reaching action generation, attenuating the signal—suggesting that the benefit of calibrated geometry depends on how directly geometry-aware tokens participate in action prediction. Tighter integration is left to future work.

4.4 Ablation Studies

We ablate G^{3}VLA's design choices on LIBERO with π_0 across three axes, summarized in Figure 3.

Geometric components. Removing ray embeddings reduces average success from 87.0% to 85.0% ($\Delta = -2.0$), the largest single-component drop. Removing PRoPE reduces it to 85.9% ($\Delta = -1.1$). Both ablations fall well below the full G^{3}VLA (π^3X) recipe, confirming that the ray embeddings and the projective cross-view structure are complementary: the ray embeddings encode per-patch viewing directions from K^{-1}, while PRoPE relates tokens across calibrated views.

Training curriculum. Replacing the two-stage curriculum with one-stage joint training reduces average success to 86.3% ($\Delta = -0.7$). The geometry-dominant pretraining phase stabilizes the new geometric pathway before the action objective dominates, and removing it incurs a measurable cost.

Supervision source. G^{3}VLA (π^3X) reaches 87.0% and G^{3}VLA (GT) reaches 88.1%, a gap of 1.1 points. GT supervision provides the strongest signal, but π^3X distillation recovers most of the gain over baseline (+2.4 points), making it a practical alternative when ground-truth depth is unavailable.

4.5 Main Real-world Results

Table 5 shows that G^{3}VLA provides the largest gains in OOD settings, where calibrated geometry becomes most important. On the pouring task, incorporating geometric priors consistently improves OOD performance across checkpoints, increasing π_0 from 70.8–75.0 to 83.3–87.5 and improving overall success from 82.5–85.0 to 90.0–92.5. Similar trends appear for $\pi_{0.5}$, where G^{3}VLA improves OOD generalization despite the stronger backbone already approaching saturation on in-distribution views. On the more spatially sensitive test-tube task, improvements are smaller but still consistent in later checkpoints, particularly for $\pi_{0.5}$ where OOD success increases from 25 to 50 at 25K, and 41.7 to 58.3 at 30K. Importantly, these gains are obtained without modifying the action space or introducing explicit 3D representations at inference. Instead, the policy benefits directly from camera-aware token geometry and projective positional structure, suggesting that calibrated multi-view information acts as an effective inductive bias for generalization under viewpoint shift.

5 Limitations

Our method assumes accurate intrinsics and extrinsics, making it sensitive to calibration drift, synchronization errors, and train–test mismatch, and it relies on a visual geometry teacher whose targets stay imperfect under occlusion, specularities, blur, or weak-prior viewpoints (gating reduces but does not remove this bias). The benefit is also architecture-dependent: on the two-tower GR00T 1.5, where the action model reaches the VLM only through cross-attention rather than consuming geometry-aware tokens, gains are attenuated. Modifying only the visual-token representation keeps the method compatible with pretrained VLAs but leaves failures rooted in the action space, limited demonstrations, or weak language–action grounding unaddressed; teacher caches and auxiliary-head training

(a) Pick-and-Place Test Tube

	π_0			G^{3}VLA (π_0+π^3X)			$\pi_{0.5}$			G^{3}VLA ($\pi_{0.5}$+π^3X)		
Chkpt.	ID	OOD	Overall	ID	OOD	Overall	ID	OOD	Overall	ID	OOD	Overall
20K	75.0	50.0	60.0	75.0	33.3	50.0	50.0	41.7	45.0	62.5	33.3	45.0
25K	62.5	41.7	50.0	75.0	41.7	55.0	37.5	25.0	30.0	50.0	50.0	50.0
30K	75.0	58.3	65.0	75.0	58.3	65.0	37.5	41.7	40.0	62.5	58.3	60.0

(b) Pouring Nut Task

	π_0			G^{3}VLA (π_0+GT)			$\pi_{0.5}$			G^{3}VLA ($\pi_{0.5}$+GT)		
Chkpt.	ID	OOD	Overall	ID	OOD	Overall	ID	OOD	Overall	ID	OOD	Overall
20K	100.0	70.8	82.5	100.0	83.3	90.0	93.75	66.7	77.5	93.75	70.8	80.0
25K	100.0	75.0	85.0	100.0	87.5	92.5	100.0	66.7	80.0	87.50	79.2	82.5

Table 5: Success rates (%) across model variants & tasks. See Appendix G for qualitative examples.

also add offline cost, though neither is needed at deployment. Future work includes online calibration robustness, broader real-world validation, and tighter integration of geometry-aware tokens with the action pathway and 3D action representations.

6 Conclusion

We presented G^{3}VLA, a camera-aware interface that injects calibrated structure into the visual-token stream of pretrained VLA policies via intrinsic-conditioned ray embeddings, PRoPE-based pose injection, bidirectional cross-view fusion, and dense point-map distillation, without changing the policy interface, action space, or imitation objective. Across LIBERO, RoboCasa24, and RoboTwin2.0, it improves the most on spatially and object-sensitive tasks, with ground-truth supervision the strongest signal and π^3X distillation a practical alternative when depth is unavailable. The results in $\pi_{0.5}$ and GR00T 1.5 further suggest that geometric transfer is most effective when geometry-aware tokens reach the action pathway directly. Overall, calibrated camera geometry is a lightweight and useful inductive bias for spatial precision in generalist VLA policies.

References

[1] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, Q. Vuong, V. Vanhoucke, H. Tran, R. Soricut, A. Singh, J. Singh, P. Sermanet, P. R. Sanketi, G. Salazar, M. S. Ryoo, K. Reymann, K. Rao, K. Pertsch, I. Mordatch, H. Michalewski, Y. Lu, S. Levine, L. Lee, T.-W. E. Lee, I. Leal, Y. Kuang, D. Kalashnikov, R. Julian, N. J. Joshi, A. Irpan, B. Ichter, J. Hsu, A. Herzog, K. Hausman, K. Gopalakrishnan, C. Fu, P. Florence, C. Finn, K. A. Dubey, D. Driess, T. Ding, K. M. Choromanski, X. Chen, Y. Chebotar, J. Carbajal, N. Brown, A. Brohan, M. G. Arenas, and K. Han. RT-2: Vision-language-action models transfer web knowledge to robotic control. In Proceedings of The 7th Conference on Robot Learning, volume 229 of Proceedings of Machine Learning Research, pages 2165–2183. PMLR, 2023. URL https://proceedings.mlr.press/v229/zitkovich23a.html.

[2] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. OpenVLA: An open-source vision-language-action model. arXiv preprint arXiv:2406.09246, 2024. doi:10.48550/arXiv.2406.09246. URL https://arxiv.org/abs/2406.09246.

[3] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky. π_0: A vision-language-action flow model for general robot control. In Proceedings of Robotics: Science and Systems, Los Angeles, CA, USA, 2025. URL https://www.roboticsproceedings.org/rss21/p010.html.

[4] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. arXiv preprint arXiv:2503.14734, 2025.

[5] M. Shridhar, L. Manuelli, and D. Fox. Perceiver-actor: A multi-task transformer for robotic manipulation. In Proceedings of The 6th Conference on Robot Learning, volume 205 of Proceedings of Machine Learning Research, pages 785–799. PMLR, 2023. URL https://proceedings.mlr.press/v205/shridhar23a.html.

[6] A. Goyal, J. Xu, Y. Guo, V. Blukis, Y.-W. Chao, and D. Fox. RVT: Robotic view transformer for 3d object manipulation. In Proceedings of The 7th Conference on Robot Learning, volume 229 of Proceedings of Machine Learning Research, pages 694–710. PMLR, 2023. URL https://proceedings.mlr.press/v229/goyal23a.html.

[7] T. Gervet, Z. Xian, N. Gkanatsios, and K. Fragkiadaki. Act3D: 3d feature field transformers for multi-task robotic manipulation. In Proceedings of The 7th Conference on Robot Learning, volume 229 of Proceedings of Machine Learning Research, pages 3949–3965. PMLR, 2023. URL https://proceedings.mlr.press/v229/gervet23a.html.

[8] D. Qu, H. Song, Q. Chen, Y. Yao, X. Ye, Y. Ding, Z. Wang, J. Gu, B. Zhao, D. Wang, and X. Li. SpatialVLA: Exploring spatial representations for visual-language-action model. arXiv preprint arXiv:2501.15830, 2025. doi:10.48550/arXiv.2501.15830. URL https://arxiv.org/abs/2501.15830.

[9] H. Zhen, X. Qiu, P. Chen, J. Yang, X. Yan, Y. Du, Y. Hong, and C. Gan. 3D-VLA: A 3d vision-language-action generative world model. arXiv preprint arXiv:2403.09631, 2024. doi:10.48550/arXiv.2403.09631. URL https://arxiv.org/abs/2403.09631.

[10] J. Zhang, A. Lin, M. Kumar, T.-H. Yang, D. Ramanan, and S. Tulsiani. Cameras as rays: Pose estimation via ray diffusion. In International Conference on Learning Representations, volume 2024, pages 23345–23366, 2024.

[11] R. Li, B. Yi, J. Liu, H. Gao, Y. Ma, and A. Kanazawa. Cameras as relative positional encoding. In Advances in Neural Information Processing Systems, volume 38, 2025. URL https://papers.neurips.cc/paper_files/paper/2025/hash/17a7075094632c88cccdd86270ad715b-Abstract-Conference.html.

[12] Y. Wang, J. Zhou, H. Zhu, W. Chang, Y. Zhou, Z. Li, J. Chen, J. Pang, C. Shen, and T. He. π^3: Permutation-equivariant visual geometry learning. arXiv preprint arXiv:2507.13347, 2025. doi:10.48550/arXiv.2507.13347. URL https://arxiv.org/abs/2507.13347.

[13] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, et al. RT-1: Robotics transformer for real-world control at scale. In Proceedings of Robotics: Science and Systems, Daegu, Republic of Korea, 2023. doi:10.15607/RSS.2023.XIX.025. URL https://roboticsproceedings.org/rss19/p025.html.

[14] Physical Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, et al. $\pi_{0.5}$: A vision-language-action model with open-world generalization. arXiv preprint arXiv:2504.16054, 2025. doi:10.48550/arXiv.2504.16054. URL https://arxiv.org/abs/2504.16054.

[15] S. Chen, R. G. Pinel, C. Schmid, and I. Laptev. PolarNet: 3d point clouds for language-guided robotic manipulation. In Proceedings of The 7th Conference on Robot Learning, volume 229 of Proceedings of Machine Learning Research, pages 1761–1781. PMLR, 2023. URL https://proceedings.mlr.press/v229/chen23b.html.

[16] T.-W. Ke, N. Gkanatsios, and K. Fragkiadaki. 3D diffuser actor: Policy diffusion with 3D scene representations. In Proceedings of the 8th Conference on Robot Learning (CoRL), volume 270 of Proceedings of Machine Learning Research, pages 1949–1974. PMLR, 2024.

[17] B. Chen, Z. Xu, S. Kirmani, B. Ichter, D. Sadigh, L. Guibas, and F. Xia. SpatialVLM: Endowing vision-language models with spatial reasoning capabilities. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pages 14455–14465, 2024. URL https://openaccess.thecvf.com/content/CVPR2024/html/Chen_SpatialVLM_Endowing_Vision-Language_Models_with_Spatial_Reasoning_Capabilities_CVPR_2024_paper.html.

[18] X. Li, L. Heng, J. Liu, Y. Shen, C. Gu, Z. Liu, H. Chen, N. Han, R. Zhang, H. Tang, S. Zhang, and H. Dong. 3DS-VLA: A 3d spatial-aware vision language action model for robust multi-task manipulation. In Proceedings of The 9th Conference on Robot Learning, volume 305 of Proceedings of Machine Learning Research, pages 2344–2359. PMLR, 2025. URL https://proceedings.mlr.press/v305/li25g.html.

[19] S. Wang, V. Leroy, Y. Cabon, B. Chidlovskii, and J. Revaud. DUSt3R: Geometric 3D vision made easy. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 20697–20709, 2024.

[20] J. Wang, M. Chen, N. Karaev, A. Vedaldi, C. Rupprecht, and D. Novotny. VGGT: Visual geometry grounded transformer. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2025.

[21] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. arXiv preprint arXiv:2010.11929, 2020.

[22] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer. Sigmoid loss for language image pre-training. In Proceedings of the IEEE/CVF international conference on computer vision, pages 11975–11986, 2023.

[23] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In Advances in Neural Information Processing Systems, volume 36, pages 44776–44791. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/8c3c666820ea055a77726d66fc7d447f-Abstract-Datasets_and_Benchmarks.html.

[24] S. Nasiriany, A. Maddukuri, L. Zhang, A. Parikh, A. Lo, A. Joshi, A. Mandlekar, and Y. Zhu. Robocasa: Large-scale simulation of everyday tasks for generalist robots. arXiv preprint arXiv:2406.02523, 2024.

[25] T. Chen, Z. Chen, B. Chen, Z. Cai, Y. Liu, Z. Li, Q. Liang, X. Lin, Y. Ge, Z. Gu, et al. Robotwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. arXiv preprint arXiv:2506.18088, 2025.

A Implementation Details

The main experiments instantiate G^{3}VLA on top of π_0 without changing the policy action space, action horizon, language conditioning, or flow-matching objective. All geometric information is introduced through the visual-token stream before the VLA projection layer.

Visual-token processing. All reported π_0 experiments use 224×224 input images. With the SigLIP patch size used by the backbone, this produces a 16×16 grid of visual patch tokens for each camera view. For each calibrated camera, we compute a dense image of normalized rays by applying the inverse intrinsic matrix to homogeneous pixel coordinates. The first two coordinates of this ray are embedded with a zero-initialized patch embedding and added to the usual image patch embedding before the SigLIP transformer layers. Thus the vision encoder receives intrinsic-aware patch tokens from its first layer, while the zero initialization preserves the pretrained visual behavior at the start of finetuning. We do not add learned camera-ID embeddings in the main model; camera identity is expressed through calibration. We also avoid geometric image augmentations that would make the RGB image inconsistent with its intrinsics or extrinsics.

After the vision encoder, patch tokens retain their view grouping. The cross-view module first applies frame attention independently within each camera stream, preserving view-local spatial structure. It then applies one global cross-view attention layer over the flattened set of valid view tokens, allowing information to flow bidirectionally between cameras. PRoPE is implemented as a dedicated pose-injection attention block inserted immediately after this global cross-view attention layer. Within that block, camera-to-world poses are converted to world-to-camera view matrices, and the intrinsics, view matrices, and patch locations define the projective transforms applied to Q/K/V. The resulting pose-conditioned attention output is added as a residual to the fused visual tokens. Invalid or padded camera views are masked throughout cross-view attention and pose injection.

Auxiliary point head. The auxiliary point head is a training-time decoder attached to the post-fusion, pre-projector visual tokens. It consumes the 16×16 patch-token grid produced by the vision backbone, but the full-resolution recipe does not compute the loss at patch resolution. Instead, it upsamples this token grid and predicts dense 224×224 point-map targets.

Concretely, the head keeps the per-view token grouping and reshapes each view's 256 patch tokens back to a 16×16 two-dimensional grid. It first maps the visual-token dimension to a hidden width of 512 and applies two transformer blocks with 8 attention heads, 2D rotary position embeddings, QK normalization, MLP ratio 4, and LayerScale initialized to 0.01. The resulting 16×16 feature grid is decoded with three transposed-convolution stages with channel widths 256, 128, and 64, followed by bilinear interpolation to 224×224. Two lightweight zero-initialized output heads predict the ray-coordinate map and the log-z map at the teacher resolution.

B Camera Geometry and Preprocessing

Camera intrinsics and extrinsics are treated as fixed inputs rather than learned parameters. Intrinsics enter the model through two pathways: they define the ray embeddings before the vision encoder, and they are used again inside the PRoPE pose-injection block. Extrinsics enter the PRoPE block as camera poses in a shared scene or robot frame.

For LIBERO simulation data, camera poses are converted to an OpenCV-style camera frame before they are used by the policy. The conversion preserves the camera center and flips the camera-frame y and z axes so that the resulting frame follows the convention (x-right, y-down, z-forward). This is the same camera-frame convention used by the ray

embedding and the point-map targets. LIBERO RGB frames are stored and evaluated in the same rotated image convention used by the training pipeline. We apply the corresponding image-space transformation to the intrinsic matrix before computing rays, so the calibrated rays remain consistent with the image seen by the vision encoder. The teacher cache is generated after the same image conversion, so cached point maps and policy inputs are spatially aligned during training.

Datasets with fewer physical cameras than the model's camera slots use padded views. These padded views are carried through the model with invalid-view masks and are excluded from cross-view attention, PRoPE pose injection, and the auxiliary geometry loss. For LIBERO, the geometry teacher supervises the base and wrist camera streams; any padded camera slot has no teacher target and does not contribute to the distillation objective.

C Geometry Teacher and Point-Map Supervision

The geometry teacher provides one precomputed full-resolution point-map target per supervised camera view and frame. Each target consists of three dense maps at 224×224 resolution: a two-channel ray-coordinate map, a one-channel log-z map, and a one-channel confidence-logit map. The ray-coordinate map represents the image-plane direction in the source camera frame. The log-z map stores $d_u^v = \log z_u^v$, where z_u^v is depth along the optical z-axis of camera v in that camera's local coordinate frame. Together, the ray coordinate and log-z value parameterize a local camera-frame point. For π^3X supervision, we use the teacher's raw local log-depth scale rather than treating the output as a reconstructed global metric point. When simulator depth is available, ground-truth point-map supervision uses the same camera-frame target schema.

Teacher target generation. For the main π^3X-supervised LIBERO recipe, teacher targets are generated offline from the converted LeRobot episodes. For each supervised camera stream, the stored RGB frames are resized to 224×224, and the camera intrinsic matrix is scaled to the same resolution. Each frame is then passed independently through the π^3X encoder, decoder, point decoder, and convolutional point head. We cache the point head's raw two-channel ray coordinate and raw pre-exponential log-z output, together with confidence logits produced by the π^3X confidence decoder and confidence head. The metric head is not added in the main four-suite target cache, so the cached log-z target represents the teacher's local depth scale rather than a globally metric reconstruction. The cache is stored at the full 224×224 output resolution; the older patch-grid mode obtained by average-pooling 14×14 pixel blocks is not used by the main full-resolution experiments.

The simulator ground-truth target variant follows the same file layout and target schema, but replaces the π^3X prediction with rendered depth. The depth image is rotated and resized to match the policy image convention, and the adjusted intrinsic matrix is used to compute $q_u^v = ((u - c_x)/f_x, (v - c_y)/f_y)$ for every pixel. Valid depth values are converted to $d_u^v = \log z_u^v$ and assigned high confidence; invalid or missing depth values receive low confidence and are removed by the confidence gate in the loss. Thus the π^3X and simulator-depth variants differ only in the source of the point-map target, not in the auxiliary-head architecture or loss interface.

The auxiliary head predicts maps at the same spatial resolution as the teacher. During training, predictions are matched only to supervised and valid camera views. Teacher confidence is used as a hard reliability gate:

$$m_u^v = M^v \mathbf{1}\left[\sigma(c_u^v) > 0.1\right], \tag{9}$$

where M^v is the view-validity indicator and c_u^v is the teacher confidence logit. The distillation objective is

$$\mathcal{L}_{\text{distill}} = \frac{\sum_{v,u} m_u^v \left(\frac{1}{2} \|\hat{q}_u^v - q_u^v\|_2^2 + (\hat{d}_u^v - d_u^v)^2 \right)}{\sum_{v,u} m_u^v + \epsilon}. \tag{10}$$

Thus confidence determines which pixels are trusted, but it does not softly reweight the regression residuals. The loss is evaluated at the full 224×224 teacher resolution. The auxiliary head and teacher targets are used only during training.

D Training Hyperparameters

We train the main π_0 model with a two-stage curriculum. Stage 1 emphasizes dense geometric supervision while preserving the pretrained action pathway: only the newly introduced ray embedding, cross-view fusion, PRoPE pose-injection block, and auxiliary point head are updated. Stage 2 starts from the final Stage 1 checkpoint, unfreezes the full policy, restores the action objective as the dominant loss, and keeps the distillation objective as a weaker regularizer.

Stage	Steps	λ_{act}	λ_{distill}	Warmup / LR
Stage 1	5k	0.1	1.0	500 steps; $2.5\times10^{-5} \rightarrow 2.5\times10^{-6}$
Stage 2	30k	1.0	0.05	1k steps; $2.5\times10^{-5} \rightarrow 2.5\times10^{-6}$

Table 6: Training schedule. Both stages use cosine learning-rate decay. Stage 1 learns the new geometry pathway under strong point-map supervision; Stage 2 fine-tunes the complete policy with a smaller auxiliary regularizer.

Both stages use AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-8}$, negligible weight decay, and global gradient clipping at 1.0. Training uses a global batch size of 32 and bfloat16 precision.

E Ablation Configuration

All LIBERO ablations use the same four-suite dataset, image resolution, optimizer, action representation, and rollout protocol as the main model. Each variant changes one part of the geometric visual-token pathway.

Base π_0 policy. The base comparison is the original π_0 policy finetuned on the same four-suite data with the standard action objective. It uses RGB observations, proprioception, and language in the original policy interface, without the calibrated visual-token module or dense point-map supervision.

No ray embedding. This variant removes the intrinsic-conditioned ray signal before the vision backbone while leaving the later multi-view fusion pathway unchanged. It isolates the contribution of early ray conditioning at the visual patch level.

No PRoPE. This variant removes the projective pose-conditioned attention used during cross-view fusion. It still receives intrinsic-aware visual tokens, so the comparison separates early ray conditioning from explicit cross-view reasoning with camera poses.

One-stage training. This variant keeps the full architecture and teacher target, but removes the geometry-adaptation stage. The model is trained end-to-end from the pretrained policy initialization with the action loss as the dominant objective and point-map prediction as a weak auxiliary regularizer.

Supervision source. The supervision-source ablation compares full-resolution point maps predicted by π^3X with point maps derived from simulator depth. Both are converted into the same ray-coordinate, log-z, and confidence format before training. We additionally use

a scale-aligned π^3X diagnostic cache to isolate how much of the gap to simulator depth comes from depth-scale mismatch rather than from the rest of the teacher prediction.

F Evaluation Details

F.1 Simulation Evaluation Protocol

All reported simulator benchmark results are averaged over three independent evaluation runs. For each run, success rates are first computed using the benchmark-specific rollout protocol, and the final reported number is the arithmetic mean over the three runs.

LIBERO. We evaluate on the four standard LIBERO suites: Spatial, Object, Goal, and LIBERO-10. Each suite is evaluated with 50 rollouts per task from the official LIBERO initial states. The environment seed is fixed to 7, and each rollout begins with 10 dummy actions to let objects settle. The maximum episode lengths are 220 steps for Spatial, 280 for Object, 300 for Goal, and 520 for LIBERO-10, matching the evaluation script.

At each policy query, the simulator renders the base and wrist cameras at 256×256. The images are rotated into the training convention and resized to 224×224 before being sent to the policy. The observation sent to the policy server contains the two RGB views, proprioceptive state, task language, and the current camera intrinsics and extrinsics. The policy predicts an action chunk, and evaluation executes the first five actions before querying the policy again. A rollout is counted as successful when the LIBERO environment returns task completion. Suite success rate is the total number of successful rollouts divided by the total number of attempted rollouts in that suite. For the four-suite LIBERO summary, we report the average over the four suite-level success rates.

For efficiency, each suite is evaluated by one policy server and multiple rollout clients. The clients shard the suite's task IDs, write per-shard JSON summaries, and the final aggregate success rate is computed by summing successes and attempted episodes across shards.

RoboCasa24 and RoboTwin2.0. For the additional simulator experiments, we use the same remote-policy evaluation pattern: a policy server receives calibrated observations and returns action chunks, while the simulator wrapper records binary task success. RoboCasa-style evaluation executes chunks of 50 actions for up to 500 environment steps and reports the mean success over completed episodes. RoboTwin evaluation follows the task wrapper's accepted-seed protocol: candidate seeds are filtered by the expert policy, the learned policy is evaluated only on accepted seeds, and success is determined by the task environment's success predicate.

Table 7 expands the aggregate RoboCasa24 result reported in the main text. Following the RoboCasa24 reporting protocol, tasks are grouped into Pick & Place, Doors / Drawers, and Others. Counts are aggregated over the same three independent evaluation runs used for the main simulator results.

Method	Pick & Place	Doors / Drawers	Others	Total Avg.
π_0 Baseline	21/160 (13.1%)	64/120 (53.3%)	79/200 (39.5%)	164/480 (34.2%)
G^{3}VLA (π^3X)	21/160 (13.1%)	63/120 (52.5%)	91/200 (45.5%)	175/480 (36.5%)
G^{3}VLA (GT)	29/160 (18.1%)	65/120 (54.2%)	84/200 (42.0%)	178/480 (37.1%)

Table 7: RoboCasa24 task-family breakdown. Success counts and rates are reported for Pick & Place, Doors / Drawers, and Others. The total average corresponds to the aggregate RoboCasa24 result reported in the main text.

F.2 Real-World Evaluation Protocol

Pick-and-Place Test Tube. The task uses two holders (source A, target B) placed at predefined locations (e.g., $A1/A2$, $B1/B3$) (see Fig. 2). We evaluate two source-target holder configurations: $A1$-$B1$ and $A2$-$B3$. For each configuration, the test tube is initialized in two different slots of the source holder, namely slot 1 and slot 3. Thus, each camera viewpoint contains four evaluation trials in total. Success is binary but allows partial credit: 1^* (grasp success, placement failure), 1 success, and 0 failure. Success rate is the fraction of successful trials.

Pouring Nut. This is a long-horizon task consisting of stacking a wheel-shaped container on on top of the red-marked base wheel and followed by pouring nuts and metal pieces into it. Four configurations are formed from two locations of the nut container ($A1/A2$) and two locations of the wheel-shaped container ($B2/B6$), with a fixed red-marked base wheel at $B5$, yielding four evaluation trials per viewpoint. Each trial is scored as 0, 0.5, 1^*, or 1, reflecting increasing task completion levels. 0.5 indicates that the robot successfully completes the stacking stage but fails to complete the pouring stage; while 1^* indicates that the robot completes the stacking and pouring stages, but some nuts are dropped outside the container. The final score averages these values.

F.3 Evaluation Stability Across Independent Runs

To assess evaluation stability, we report the variability of the main π-series experiments over three independent evaluation runs. This analysis excludes the diagnostic ablations, which are intended to isolate architectural components rather than estimate run-to-run variance. For each method and benchmark, we report mean $\pm$ standard deviation over the three evaluation runs. For LIBERO, the average row is computed by first taking the macro-average over the four suites within each run and then computing the standard deviation across runs.

Suite	π_0 Baseline	G^{3}VLA (π^3X)	G^{3}VLA (GT)
Goal	87.4$\pm$0.5	88.4$\pm$0.6	88.4$\pm$0.4
Spatial	85.2$\pm$0.7	88.6$\pm$0.6	89.2$\pm$0.5
Object	89.4$\pm$0.5	93.4$\pm$0.5	94.4$\pm$0.4
L-10	76.5$\pm$0.9	77.6$\pm$0.8	80.4$\pm$0.7
Average	84.6$\pm$0.4	87.0$\pm$0.4	88.1$\pm$0.3

Table 8: Three-run variability on LIBERO with π_0 backbone. Values are success rates (%) reported as mean $\pm$ standard deviation over three independent evaluation runs.

Benchmark	Method	Success
RoboCasa24	π_0 Baseline	34.2$\pm$0.9
	G^{3}VLA (π^3X)	36.5$\pm$1.1
	G^{3}VLA (GT)	37.1$\pm$1.0
RoboTwin2.0	π_0 Baseline	44.0$\pm$2.0
	G^{3}VLA (π^3X)	41.0$\pm$1.7
	G^{3}VLA (GT)	49.0$\pm$2.5

Table 9: Three-run variability on broader π_0 simulation benchmarks. RoboCasa24 reports the total average success rate; RoboTwin2.0 reports the handover-block diagnostic. Values are success rates (%).

F.4 Teacher-Target Failure Case on RoboTwin2.0

RoboTwin depth-teacher failure case. RoboTwin contains visually clean, texture-sparse simulator scenes. This makes it a useful stress test for geometry-aware policies that rely on a monocular foundation model teacher. In the `handover_block` task we therefore compare the Pi3X teacher cache against simulator ground-truth depth, using the same camera views and frame indices used by policy training. Because the student is trained on the raw Pi3X `log_z` targets, we select the frame with the largest mean absolute log-ratio between the Pi3X median depth and the simulator-GT median depth across the three cameras. We also report a scale-invariant shape error, computed after robustly normalizing each log-depth map within the frame, to separate global scale mismatch from relative geometry.

Figure 4 shows the automatically selected example: episode 24, frame 227. The error is most severe in the right wrist camera: Pi3X predicts a median depth of 3.535m while the simulator GT median is 0.027m, a 132.4× mismatch. Averaged over the three cameras, the selected frame has mean absolute log-ratio 3.56 and scale-invariant MAE 0.17. Thus the auxiliary target is geometrically miscalibrated in the clean simulator domain even when the RGB image is unambiguous and ground-truth depth is available.

This explains the negative transfer observed for Pi3X-distilled RoboTwin policies: the auxiliary target can inject a confident but geometrically wrong prior in the clean simulator domain. Replacing the monocular teacher with simulator ground-truth depth removes this source of target noise, which is why the GT-only variant improves success rate even though the Pi3X-distilled variant can have lower training losses.

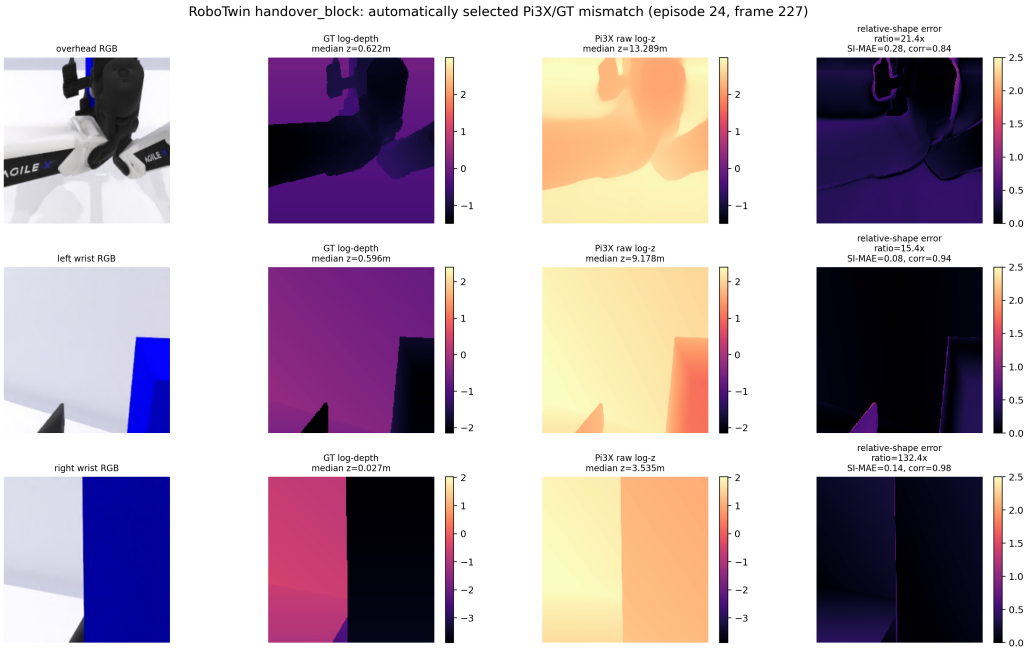


Figure 4: Worst automatically selected Pi3X/GT depth mismatch in RoboTwin `handover_block`. GT and Pi3X log-depth panels use the same color scale per view. The right column shows the remaining relative-shape error after per-frame robust log-depth normalization.

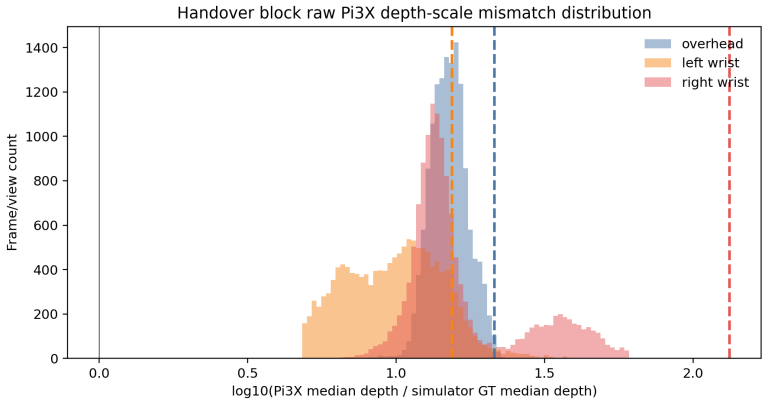


Figure 5: Distribution of raw median-depth scale mismatch across the handover-block cache. Zero means Pi3X and simulator GT have the same median depth. Dashed vertical lines mark the three views in the selected failure frame.

G Qualitative Examples & Failure Cases

Recovery Behavior in Real Robot Experiments. The GT-based variants, including G^{3}VLA with π_0+GT and $\pi_{0.5}$+GT, show robust recovery behavior during the long-horizon pouring nut task. During stacking, if the wheel-shaped container is slightly misaligned on the red-marked base, the robot often re-grasps the container and adjusts its pose before continuing (Figure 6). During pouring, if the first grasp of the blue container fails or the container slips, the robot can attempt to re-grasp it multiple times within the same rollout (Figure 7).

This recovery behavior is also observed under OOD camera viewpoints. The robot may initially move toward an incorrect grasp position where the blue container is not visible in the wrist camera, but it can recover by repositioning, using the context camera, or moving upward until the container appears in the wrist-camera view. It then moves toward the visible container and attempts the grasp again (Figure 8). In contrast, these recovery behaviors were not observed in the vanilla models. These observations suggest that GT-based geometry supervision improves robustness by enabling corrective actions after intermediate errors.

Failure Cases. Across both tasks, we observe a common failure mode for all models where the robot moves toward an incorrect predefined object location (Figure 9). In the test tube task, the robot sometimes overfits to source slot 3 and moves there even when the tube is placed at slot 1, for both ID and OOD camera viewpoints. In the pouring nut task, a similar location-overfitting failure occurs mainly under OOD viewpoints, where the robot moves toward the wrong predefined location to grasp the blue container. We also observe failures caused by unstable or incorrect grasp poses, where the object slips or is grasped from an unsuitable side (Figure 10).

Since these failures appear in both vanilla baselines and model variants, they likely reflect general VLA failure modes rather than issues specific to our method. However, as described above, the GT-based variants can often recover from these failures through corrective behavior.

Figure 6: Alignment correction during the pouring nut task. (a) The wheel-shaped container is initially placed on the red-marked base with slight misalignment. (b) The GT-based model corrects the placement by re-grasping and adjusting the container to achieve better alignment before continuing to the pouring stage.

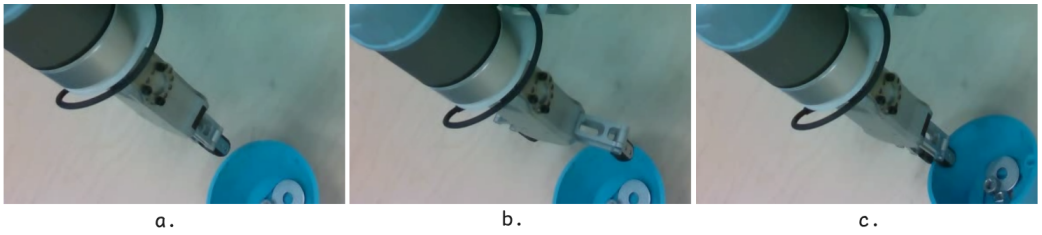


Figure 7: Re-grasping behavior during the pouring nut task. (a) The robot fails to grasp the blue container on the first attempt. (b) The robot opens the gripper and repositions for another grasp attempt. (c) The robot successfully re-grasps the blue container and continues the task.

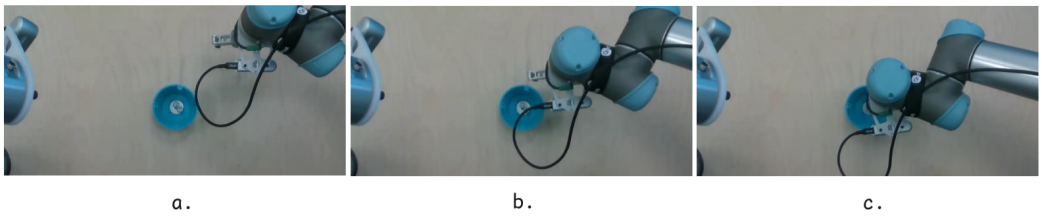
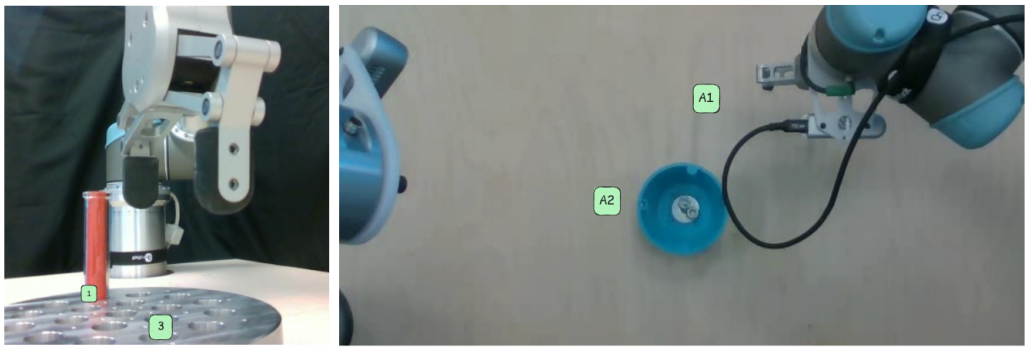
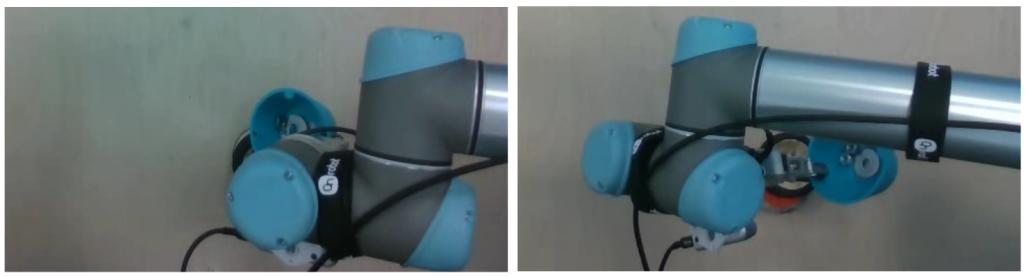


Figure 8: Recovery from location-overfitting under an OOD camera viewpoint. (a) The robot initially moves toward an overfitted grasp location instead of the blue container. (b) During recovery, the robot repositions and searches for the blue container. (c) Once the blue container appears in view, the robot moves toward the correct position and successfully grasps it.



a. b.

Figure 9: Location-overfitting failure cases across the two tasks. (a) In the test tube task, the robot often moves toward source slot 3 even when the tube is initialized at a different predefined slot. (b) In the pouring nut task, especially under OOD camera viewpoints, the robot can move toward the predefined $A1$ location when attempting to grasp the blue container, even when the container is placed at $A2$.



a. b.

Figure 10: Incorrect grasp-side failure in the pouring nut task. (a) The robot grasps the blue container from a suitable side, enabling it to lift and pour the contents. (b) The robot grasps the blue container from the wrong side, which prevents a successful pouring motion.